

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Graph-aware Large Language Models (LLMs) extend graph learning beyond conventional Graph Neural Networks (GNNs) by jointly modeling relational structure and textual node attributes. Although these models are increasingly used for graph tasks, their adversarial robustness has remained largely uncharacterized. This paper presents the first systematic study of attacks and defenses for graph-aware LLMs, evaluating representative poisoning (training-time) and evasion (test-time) graph attacks on two projector-based models, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). The study further identifies a graph-aware attack surface specific to LLAGA: adversarial nodes can be inserted into placeholder positions in the node sequence template, causing substantial degradation. Across datasets, attack types, and perturbation channels, the results establish graph encoding design as a primary determinant of robustness: (1) the node sequence template in LLAGA amplifies sensitivity to structural manipulation; (2) the GNN encoder in GRAPH PROMPTER provides comparatively stronger resistance to structural perturbations; and (3) both projector paradigms remain highly vulnerable to visually imperceptible perturbations of textual node features. Finally, we introduce GALGUARD, an end-to-end defense framework that couples LLM-based feature correction with adapted GNN defenses for structural perturbations¹

1 Introduction

Graphs represent entities and their relations as nodes and edges, making them a central abstraction for complex relational systems in social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018) have therefore become a standard paradigm for graph-structured tasks, including node classification and link prediction. The rapid development of large language models (LLMs), including GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), creates a complementary modeling opportunity: large-scale text models can be adapted to exploit the textual attributes and relational organization that jointly appear in

many graph datasets. This integration, referred to as *graph-aware LLMs* (Xie et al. 2023), has emerged as an important direction in which LLMs address tasks previously dominated by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Recent results show that graph-aware LLMs can outperform GNNs on several graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), establishing them as substantive graph-learning models rather than auxiliary language modules.

This expanding role makes robustness a necessary property of graph-aware LLMs, yet their adversarial behavior has not been systematically established. Existing attacks on GNNs, including poisoning (training-time) and evasion (test-time) attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), typically assume a black-box adversary without access to the target model’s architecture or parameters. These attacks compromise *model integrity* by introducing subtle structural or feature perturbations that reduce predictive performance. Graph adversarial research has primarily emphasized structural manipulation; graph-aware LLMs enlarge this attack surface by introducing a textual interface through which visually imperceptible changes to node attributes can alter the token sequence consumed by the LLM while remaining innocuous to human readers.

These observations lead to the central question of this paper: *do adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm*, and what additional vulnerabilities follow from the mechanisms used to encode graphs for LLMs? The answer depends on the interface through which graph information reaches the language model. Textual node features are naturally compatible with LLM inputs, whereas graph structure must be transformed by an encoding mechanism that preserves relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore organize graph-aware LLMs into two encoding classes: methods that express graphs as *textual descriptions*, and methods that use *learned projectors* to map graph information into representations consumed by a frozen LLM. Because textual-description methods inherit well-studied text attack surfaces and often encounter scalability constraints, our empirical analysis focuses on learned

¹Our code will be released upon acceptance.

projector methods, represented by two computationally efficient models, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024), both of which retain frozen pre-trained LLM backbones.

This study establishes graph encoding design as a decisive determinant of adversarial robustness in graph-aware LLMs. We identify a new attack surface in LLAGA: by manipulating the node sequence template so that malicious nodes occupy placeholder positions, an attacker can substantially degrade performance. We further show that feature perturbations, often secondary in conventional GNN settings, become a major vulnerability once textual node attributes are processed by LLMs. To complement the attack analysis, we propose GALGUARD, an end-to-end graph-aware LLM defense that combines LLM-based feature correction with adapted GNN-based structural defenses. To our knowledge, this is the first systematic investigation of adversarial attacks and defenses for graph-aware LLMs. The paper makes five connected contributions:

- We systematically evaluate the transferability of representative GNN attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to graph-aware LLMs under poisoning and evasion settings.
- We provide a graph encoding-based taxonomy of graph-aware LLMs that clarifies how different methods expose graph structure to a frozen LLM and motivates the focus on projector-based models.
- We identify and evaluate a LLAGA-specific attack surface in which malicious nodes are injected as placeholders in the node sequence template, revealing a vulnerability introduced by graph-to-sequence encoding.
- We demonstrate that imperceptible textual feature perturbations, implemented through Homoglyph and Reordering attacks, substantially degrade graph-aware LLM performance across evaluated datasets and models.
- We introduce GALGUARD, a defense framework that jointly addresses textual feature perturbations and structural graph perturbations through LLM-based correction and adapted GNN defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph-aware LLM methods are most clearly distinguished by the mechanism used to translate graph-structured information into a form consumable by a language model. This encoding choice is central to the robustness problem studied in this paper: it determines which graph components become visible to the LLM, how structural and textual signals interact, and how adversarial perturbations propagate to the model input. As illustrated in Figure 5, existing approaches fall into two principal categories: *textual descriptions of graphs*, which serialize graph information as natural language, and *learned graph projectors*, which map graph data into embeddings or structured token-level representations. Appendix A.1 expands this taxonomy, and Appendix A.2 reviews the corresponding literature on GNN attacks.

Graph Representation as a Textual Description. Textual-description approaches convert graph data into natural language so that LLMs can process relational information through prompts and instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal settings, Liu et al. (2023) and Su et al. (2022) associate molecular graphs with textual descriptions. A related line of work studies how graph structure should be incorporated into prompts; for example, Huang et al. (2023a) shows that LLMs often interpret graph prompts as contextual paragraphs rather than explicit graph objects, making prompt design a substantive modeling decision (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Additional methods use LLMs to enrich node features or graph structures, operate as predictors, or provide feature representations for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) combine LLMs and GNNs for textual graphs, and Duan et al. (2023) introduce an effective textual graph learning approach. These methods offer a direct route for applying LLMs to graph tasks, but they can fail to preserve fine-grained structural information, particularly in large graphs, and remain sensitive to prompt engineering choices.

Graph Representation as a Learned Projector. Learned-projector methods address these limitations by transforming graph data into embeddings, graph vectors, or token-like structures that can be aligned with the LLM input space. Representative approaches include LLAGA, which constructs node-level templates and applies a linear projector (Chen et al. 2024), and methods that incorporate GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Other variants use graph neural prompts to encode knowledge graphs (Tian et al. 2024), prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or graph transformers that compress graph information into fixed-length prefixes (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024). Because projector methods expose an explicit graph-to-LLM interface, they provide a precise empirical setting for determining how encoding design mediates adversarial perturbations.

This Work: We focus on projector methods built on *frozen pre-trained LLMs* because they provide computationally efficient graph adaptation while avoiding the substantial cost of retraining the language model (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). The empirical study uses two representative designs: LLAGA (Chen et al. 2024), which relies on node-level templates and a linear projector, and GRAPH-PROMPTER (Liu et al. 2024), which combines a GNN encoder with a linear projector. This choice isolates the robustness implications of learned graph projectors. Textual-description methods remain outside the empirical scope because their scalability limitations and conventional text-prompt attack surfaces are less aligned with the structural vulnerabilities examined here.

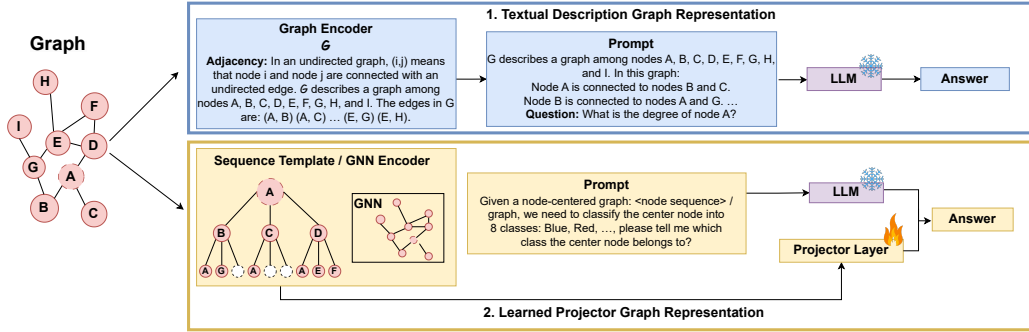


Figure 1: **Graph encoding-based adaptations of LLMs.** Textual-description methods express graph information as natural language (top), whereas learned-projector methods encode graphs through node templates that convert neighborhoods into sequences or through GNN encoders (bottom). In both projector cases, the LLM remains frozen and only the graph-to-token projector is trained.

3 Threat Model

We define the threat model by extending established graph adversarial settings to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The setting captures realistic limited-control scenarios in which an attacker cannot inspect the target architecture or parameters but can still perturb graph data. In a social network, for example, an attacker may control only a small number of bot accounts while manipulating local graph structure or textual attributes to evade detection or influence the classification of legitimate users. In a Wikipedia-like graph, sparsely connected hoax articles may be strategically linked to authentic articles to induce misclassification of genuine content. These threats are especially consequential as graph-aware LLMs are considered for recommendation, social media analysis, healthcare, and other domains in which they may supplement or replace GNN-based pipelines. Following prior work, we specify the threat model through model access, adversarial capabilities, attack strategies, and the attacker’s objective.

Model Access. The attacker operates in a black-box setting and has no knowledge of the target model’s architecture or parameters. The attacker can influence the model only through the training data or through test-time inputs.

Adversarial Capabilities. Consistent with poisoning and evasion attacks, respectively, the attacker has access to either the training data or the inference-time data of the target model. Under a predefined perturbation budget and unnoticeability constraint, the attacker may modify existing data or inject additional data.

Attack Strategies. The attacker uses two perturbation channels: feature manipulation, which alters node attributes, and structural manipulation, which adds or removes edges.

Attacker’s Goal. The attacker seeks to compromise model integrity by degrading overall classification performance, operationalized as an increased misclassification rate.

4 Experimental Setup

Dataset. We evaluate graph-aware LLM robustness on three widely used text-attributed graph benchmarks: Cora, CiteSeer, and PubMed (Yang, Cohen, and Salakhudinov 2016). Together, these datasets cover small-to-medium graph scales and distinct sparsity regimes, providing standard evaluation conditions for the projector-based graph-aware LLMs studied here (Chen et al. 2023a, 2024; Liu et al. 2024). To assess whether the observed vulnerability patterns persist at larger scale, we additionally report an ArXiv experiment (Hu et al. 2020) in Appendix A.5. Following (Chen et al. 2024), we use train/validation/test splits of 6:2:2 for Cora, CiteSeer, and PubMed, and 5:2:3 for ArXiv. Table 1 summarizes the graph statistics for the main evaluation.

Table 1: Statistics of the text-attributed graph benchmarks used in the main robustness evaluation.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We use *node classification* as the primary evaluation task because it is a canonical graph machine learning benchmark and the standard setting in which the poisoning and evasion attacks examined here are defined (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report *accuracy*, defined as the proportion of correctly classified nodes. Each experiment is repeated three times, and all tables report the mean and standard deviation. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We select graph-aware LLMs and attack methods to cover representative projector architectures and established graph adversarial mechanisms, aligning the empirical scope

Table 2: Node classification accuracy of LLAGA and GRAPHPROMPTER under NETTACK and METAATTACK on Cora, Citeseer, and PubMed. Percentages in parentheses report the relative drop from clean accuracy under the corresponding attack setting.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

with both the emerging graph-aware LLM literature and the standard graph-attack setting.

Graph-Aware LLM Methods. We evaluate two representative graph-aware LLMs: LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). Both retain a *frozen pre-trained LLM*, enabling computationally efficient adaptation across graph tasks and LLM architectures. LLAGA constructs node-level templates and maps them through a linear projector, whereas GRAPHPROMPTER first encodes graph information with a GNN-based encoder and then applies a linear projector. In both models, the projector is a 2-layer multi-layer perceptron (MLP). The base LLMs for LLAGA and GRAPHPROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We evaluate robustness using poisoning and evasion variants of two representative graph adversarial attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019). NETTACK is a targeted attack that identifies minimal perturbations sufficient to misclassify selected nodes, whereas METAATTACK performs an untargeted attack by optimizing graph structure as a hyperparameter. For NETTACK, we follow (Li et al. 2023; Zügner and Günnemann 2019): each node is targeted sequentially and the resulting outcomes are aggregated, yielding an untargeted evaluation protocol. Across experiments, we perturb only 10% of edges, preserving the unnoticeability constraint used in prior work (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Appendix A.2 provides additional attack details.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We evaluate poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to graph-aware LLMs. For poisoning attacks on LLAGA, the perturbed graph is first converted into node-level sequences and the projector is then retrained. For poisoning attacks on GRAPHPROMPTER, the perturbed graph is re-encoded with a GAT before projector retraining. In evasion attacks, both models are trained on clean data, and perturbations are introduced only at inference time.

Results. Table 2 establishes three empirical patterns that organize the subsequent analysis. First, poisoning attacks produce limited degradation in this evaluation, with a maximum reported drop of 11%, whereas evasion attacks are substantially more damaging and reach a 51% drop. This contrast indicates that projector training in the presence of perturbations, together with the frozen LLM context, provides greater resilience than inference-time exposure to previously unseen corruptions. Second, METAATTACK generally causes larger accuracy reductions than NETTACK under both poisoning and evasion, consistent with its optimization of graph edges as hyperparameters and its ability to identify small but consequential structural changes. Third, the observed structural vulnerability is data-dependent: for example, LLAGA drops by 51% on the denser Cora graph under METAATTACK evasion, compared with 19% on the sparser PubMed graph.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

The comparison in Table 2 shows that LLAGA is more susceptible than GRAPHPROMPTER because its representation is more directly coupled to perturbed graph structure. LLAGA incorporates structure by constructing a fixed-length node-level sequence for each node and computing a graph Laplacian embedding. Each sequence position encodes a relative structural role in the original graph; consequently, structural perturbations alter the node embeddings delivered to the LLM. GRAPHPROMPTER, by contrast, preserves textual node embeddings under structural perturbation because the attack affects the GNN component rather than the text representation. This separation allows unperturbed textual features to partially compensate for corrupted structural information, yielding stronger robustness in the evaluated settings.

6 Node Sequence Template Injection Attacks

We identify a new attack surface in LLAGA by exploiting the neighborhood detail template used to construct node sequences. Figure 2 illustrates the mechanism, and Appendix A.4 specifies the algorithm. To obtain the node sequence for a node u , LLAGA constructs a *fixed-shape* computational tree whose children are sampled from 1-hop neighbors; the procedure is then repeated at subsequent tree levels. When a node has too few neighbors, placeholders are inserted to

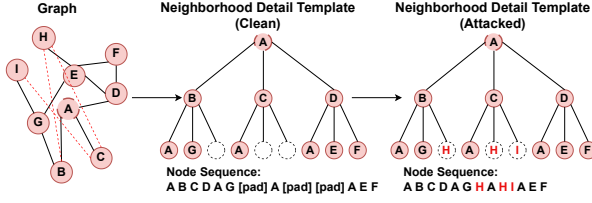


Figure 2: Node sequence template injection attacks. The example shows Non-adjacent Injection (NI), where red dashed lines denote attacker-induced edge perturbations. In Supernode Injection (SI) and Multiple Supernode Injection (MSI), the injected supernode or supernodes occupy placeholder positions in the fixed-width template. Here, $k = 3$.

preserve the fixed width, such as $k = 10$ children in the original LLAGA design (see "Clean" from Figure 2 and lines 1-14 in the algorithm). This placeholder mechanism converts a representational convenience into a graph-specific injection surface.

We instantiate this vulnerability through three injection strategies. *Non-Adjacent Injection (NI)* inserts nodes that lie outside the 2-hop neighborhood of a target node u (see "Attacked" from Figure 2 and lines 21-27). *Supernode Injection (SI)* inserts a high-degree node as a placeholder while keeping it outside the target node's 2-hop neighborhood. *Multiple Supernode Injection (MSI)* extends *SI* by injecting multiple distinct supernodes as placeholders (lines 35-43). In each case, uninformative or adversarially selected nodes enter the node sequence and corrupt the target representation, degrading the projector during training or inference.

Results. As shown in Table 3, LLAGA remains comparatively resilient to poisoning on Cora and Citeseer but is sharply degraded under evasion. On Cora, accuracy decreases to 0.42, 0.36, and 0.30 under *NI*, *SI*, and *MSI*, respectively; on Citeseer, the corresponding accuracies are 0.36, 0.30, and 0.24. These results show that the fixed-width sequence template creates a concrete inference-time vulnerability rather than merely an implementation detail.

On the larger PubMed dataset, LLAGA exhibits stronger resilience to all injection attacks, with evasion accuracies of 0.85, 0.82, and 0.76. This pattern is consistent with the structural scale of the datasets: in smaller and more tightly connected graphs such as Cora and Citeseer, a localized injection can more strongly disrupt neighborhood reasoning, whereas in the larger PubMed graph the same perturbation is comparatively diluted.

Relative to the traditional graph adversarial baselines, the node injection attacks expose a stronger weakness in LLAGA on Cora and Citeseer. For example, *MSI* yields evasion accuracies of 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, whereas METAATTACK yields 0.44, 0.50, and 0.73 under the same datasets (Table 2). The comparison demonstrates that attacks tailored to the graph-to-sequence interface can be more damaging than attacks transferred directly from conventional GNN robustness settings.

Table 3: Accuracy of LLAGA under node sequence template injection attacks. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively; Clean reports performance without attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks primarily target structure (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019), with the exception of NETTACK (Zügner, Akbarnejad, and Günnemann 2018), which does not directly transfer to the textual domain. Because structural attacks are only moderately successful on comparatively robust models such as GRAPH PROMPTER, and because textual node attributes contribute substantially to graph-aware LLM predictions, we evaluate unnoticeable feature perturbation attacks as a distinct and central robustness threat.

We adopt the imperceptible attack strategies proposed by (Boucher et al. 2022) with one key modification: perturbations are generated using a task-agnostic translation objective (English to French). The objective modifies input text so that changes remain visually similar while inducing an incorrect translation, thereby corrupting the encoded semantics. The premise is that a perturbation capable of disrupting translation also disrupts the semantic representation available to the downstream model. We evaluate two imperceptible attacks: *Homoglyphs*, which replace characters with visually similar alternatives, and *Reorderings*, which use direction control characters to alter the rendering sequence. Examples of Homoglyphs and Reorderings are in Appendix A.6.

Following (Boucher et al. 2022), we optimize these perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between the model's outputs on perturbed and unperturbed text. The perturbation budget is set to 10% of the average length of each textual feature.

Results. As shown in Table 4, Reordering attacks are generally more damaging than Homoglyph attacks across the evaluated models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both attack types: poisoning attacks cause accuracy drops of 27% and 23% for Homoglyph, and larger drops of 35% and 25% for Reordering, respectively. In both cases, GRAPH PROMPTER is more resilient. PubMed follows the same qualitative pattern, although the overall degradation is less pronounced than on the smaller datasets.

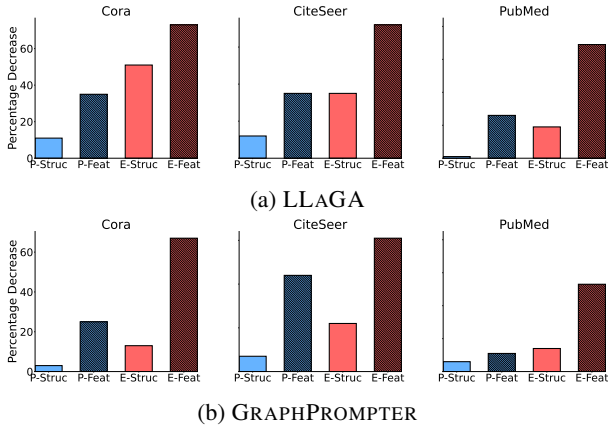


Figure 3: Relative degradation caused by structural and feature-based adversarial attacks on graph-aware LLMs. Attack success is measured as the percentage decrease from clean accuracy. P-Struc, P-Feat, E-Struc, and E-Feat denote poisoning and evasion attacks based on structural and feature perturbations.

The comparison with structural perturbations in Figure 3 reveals a clear shift in the vulnerability profile. Feature perturbation attacks, especially Reordering, outperform structural attacks based on METAATTACK by a large margin on both LLAGA and GRAPHPROMPTER. This pattern establishes textual feature integrity as a central robustness requirement for the evaluated graph-aware LLMs. The next section therefore evaluates a unified attack that perturbs both structure and features.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We propose a unified attack that combines feature and structural perturbations into a coordinated adversarial strategy. The attack is designed to degrade model accuracy more strongly than either perturbation channel alone while preserving inconspicuous changes to both graph structure and textual features.

For the structural component, we use METAATTACK, which modifies graph structure through subtle edge adjustments. For the feature component, we combine Homoglyph and Reordering attacks as described in the previous section. Together, these perturbations target the relational and textual channels on which graph-aware LLMs jointly rely.

Results. As shown in Table 5, jointly applying structural and feature perturbations produces substantially stronger degradation than either perturbation channel alone across the evaluated datasets. The unified attack reduces accuracy by 53%–69% under poisoning and by 79%–87% under evasion, demonstrating that graph-aware LLM robustness depends on the joint consistency of relational structure and textual node attributes. Unlike traditional GNN settings, where structural perturbations dominate, the evaluated graph-aware LLMs expose both graph-level and feature-level adversarial channels.

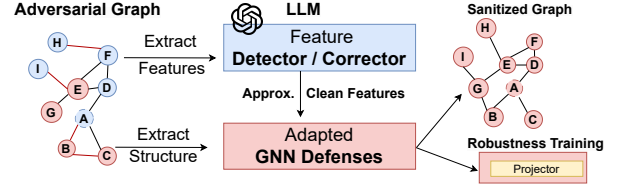


Figure 4: **Overview of GALGUARD.** Perturbed node features and edges are shown in blue and red, respectively. The LLM identifies perturbations in node features extracted from an adversarial graph and generates a corrected version. The refined features, together with the perturbed edges, are then processed by existing GNN defenses, which can be used either to obtain a sanitized graph or to support projector robustness training.

9 Ablation Study

9.1 Varying Feature Perturbation

To characterize the relationship between perturbation budget and attack effectiveness, we vary the percentage of perturbed textual features. The Reordering attack, which follows patterns similar to the Homoglyph attack, shows a consistent trend as the perturbation percentage varies from 1% to 10%: for all evaluated models and datasets (Cora, Citeseer, and PubMed), lower perturbation rates yield weaker attacks. As the perturbation level decreases, poisoning and evasion success rates tend to converge because the adversarial signal becomes too weak to substantially affect the model, regardless of whether it is introduced during training or inference. A detailed plot illustrating these trends is in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We evaluate whether the foundation LLM changes adversarial robustness in graph-aware LLMs. LLAGA uses Vicuna-7B (Chiang et al. 2023), whereas GRAPHPROMPTER uses Llama2-7B (Touvron et al. 2023). We replace LLAGA’s backbone with Llama2-7B and compare performance under poisoning and evasion attacks on all datasets. Both backbones achieve similar clean accuracy (e.g., Vicuna: 0.89 ± 0.07 , Llama2: 0.90 ± 0.03 on Cora) and exhibit comparable vulnerability under attack. The same trend holds when swapping LLMs in GRAPHPROMPTER. These results indicate that, within the evaluated settings, graph encoding and perturbation channel have a stronger effect on robustness than the particular foundation LLM. The full results are in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses do not directly address the robustness profile of graph-aware LLMs because these models are jointly vulnerable to structural and textual feature perturbations. Many graph defenses prune or reweight edges using node feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), but

Table 4: Accuracy under feature perturbation attacks, with percentage drop from clean accuracy in parentheses.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

they assume that node features remain reliable. When textual features are adversarially perturbed, these defenses can lose the similarity signal on which they depend (see Figure 3 in the Appendix). Similarly, defenses that decouple structure and features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021) and OCR-based sanitization strategies (Boucher et al. 2022) are insufficient under coordinated attacks. The attack results therefore motivate an integrated defense that treats feature correction and structural robustness as coupled requirements.

To address this need, we propose graph-aware LLM defense (GALGUARD), an end-to-end defense strategy with two complementary components: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As shown in Figure 4, the LLM corrector acts as an adaptive mechanism for identifying anomalous patterns in textual node features. It generates approximate corrections intended to restore feature integrity and reduce the effect of feature-level perturbations. This component directly targets the vulnerability exposed by imperceptible textual attacks. The specific instructions and prompts are in Appendix A.6. For our experiments, we employ GPT-4 Turbo as the LLM corrector.

Graph Structure Robustness Strategy. To improve structural robustness, we use a two-step strategy. First, we adopt a *graph purification heuristic* (Wu et al. 2019) as preprocessing. For each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and the embeddings $\phi(x_u)$ of its neighbors, and remove edges with low similarity. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). For GRAPHPROMPTER, we directly apply the GNNGuard principle to its message-passing layers. For LLAGA, we adapt the defense to the sequence-construction interface by incorporating purification into the computational tree used to build node sequences. We further introduce a learnable *global structural context embedding* (M_{global}) as a parameter in LLAGA’s projector. By concatenating M_{global} with the purified node sequence embedding and jointly training them, the model learns to operate on potentially perturbed graph inputs. The plug-and-play design of GALGUARD allows other structural defenses, such as Pro-GNN (Jin et al. 2020), to replace GNNGuard.

Table 5: Model accuracy under the unified imperceptible attack. Percentages in parentheses report the drop from clean accuracy.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

10.1 Results

Table 6 reports accuracy under METAATTACK and under two defense variants: GALGUARD_p, which combines feature correction with graph purification, and the full GALGUARD, which additionally integrates GNNGuard or its LLAGA-specific adaptation. Across Cora, CiteSeer, and PubMed, and under both poisoning and evasion settings, the proposed defenses mitigate the effect of METAATTACK. GALGUARD_p already improves robustness, showing that LLM-based feature correction and initial graph purification form an effective foundation. The full GALGUARD consistently provides the strongest results, indicating that feature correction and GNN-inspired structural defense reinforce one another.

Table 6: Accuracy under METAATTACK and the proposed defenses. GALGUARD_p combines feature correction with graph purification; GALGUARD additionally incorporates GNNGuard or its LLAGA-specific adaptation.

Data	Method	LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

This paper establishes adversarial robustness as a central concern for graph-aware LLMs and provides the first systematic study of attacks and defenses in this setting. The results show that the graph encoding interface is a primary source of vulnerability: LLAGA’s node sequence template creates an injection surface through placeholder positions, while GRAPHPROMPTER’s GNN-based encoding provides comparatively stronger resistance to structural perturbations. The experiments further demonstrate that imperceptible textual feature perturbations are highly effective against graph-aware

LLMs, often exceeding the impact of traditional structural attacks, and that a unified attack combining both channels can produce severe degradation.

We address these vulnerabilities with GALGUARD, an end-to-end framework that integrates LLM-based feature correction with adapted GNN-based structural defenses. As graph-aware LLMs are increasingly considered for graph learning settings in domains such as social networks, health-care, and finance, their reliability depends on defenses that account for both textual and relational attack surfaces. The findings in this paper provide a foundation for that agenda by clarifying where current graph-aware LLM designs are vulnerable, how those vulnerabilities differ from conventional GNN robustness challenges, and how integrated defenses can begin to mitigate them.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.

- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejdl, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.

- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix provides supporting technical detail for the main analysis. Appendix A.1 expands the taxonomy of graph-aware LLMs. Appendix A.2 reviews attacks against Graph Neural Networks (GNNs). Appendix A.3 summarizes NETTACK and METAATTACK, and Appendix A.4 specifies the node sequence injection attack on LLAGA. Appendix A.5 reports ablations on feature perturbation level and base LLM choice. Appendix A.7 provides feature-attack examples and the LLM corrector template, and Appendix A.8 discusses the broader implications of the results.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data has produced a broad set of methods for graph reasoning, representation, and inference. The key distinction among these methods is the graph encoding interface: graph information is either converted into *textual descriptions of graphs*, where topology and attributes are expressed in natural language, or passed through a *learned graph projector*, where graph data are transformed into embeddings or structured representations that the LLM can process. This section details both encoding families. Figure 5 illustrates the two approaches, and Table 7 summarizes their operational differences.

Graph Representation as a Textual Description In the textual-description family, graph data are represented as natural language so that LLMs can process graph structures through prompts or instructions. This strategy has been developed in work that converts graph data into textual formats for LLM querying (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a). For example, Tan et al. (2023) introduced a unified framework for attributed graph embedding by generating random-walk textual sequences from graphs and using them to fine-tune LLMs. Similarly, Zhao et al. (2023) translated graph structures into natural language to support graph reasoning in text spaces. Extending this direction, Wei et al. (2024a) augmented textual graph representations with visual graph representations to improve LLM reasoning. In multimodal applications, Liu et al. (2023) and Su et al. (2022) linked molecular graphs with textual descriptions for drug discovery and molecular retrieval. Related prompt-oriented work shows that graph prompts are often processed as contextual paragraphs rather than explicit graph objects (Huang et al. 2023a), reinforcing the importance of prompt design in heterogeneous and large-scale graph tasks (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). Complementary work uses LLM-generated explanations as features to improve performance and interpretability on graphs (He et al. 2024).

A related line of textual-description work combines LLMs with GNNs through pretraining or feature construction. For

instance, Chen et al. (2023a) proposed two LLM-GNN integration approaches: using LLMs to enhance node features for node classification, and employing LLMs as direct predictors for graph tasks. In a similar direction, Chandra et al. (2020) used LLMs as text encoders and GNNs as graph-structure encoders for fake-news detection in online communities. Chen et al. (2023b) used LLMs to annotate nodes for label-free node classification, after which the generated labels are used to train a GNN model, and Mavromatis et al. (2023) used LLMs as feature extractors for GNNs.

For inference, Zhu et al. (2024) proposed a textual-graph fine-tuning method that combines LLMs and GNNs through a tunable side structure at each LLM layer, improving both training efficiency and inference speed. Similarly, Duan et al. (2023) introduced SimTeG, which enhances textual graph learning by first fine-tuning the LLM on a downstream task such as node classification and then extracting node embeddings from the final hidden states. Additionally, Zhang (2023) proposed Graph-toolformer, which uses ChatGPT-powered prompt engineering to augment LLM graph reasoning.

Textual-description approaches make LLMs readily applicable to graph tasks, but they are constrained by the fidelity and scalability of natural-language graph serialization. They may fail to preserve structural detail, require careful prompt engineering, and become difficult to apply when larger graphs exceed prompt-length budgets.

Graph Representation as a Learned Projector Learned-projector methods provide a more explicit graph-to-LLM interface by transforming graph data into embeddings, graph vectors, or graph tokens. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that preserves the general-purpose capabilities of LLMs while converting graph structures into graph tokens suitable for LLM processing. Similarly, (Liu et al. 2024; Qin et al. 2023) incorporated graph structure into a pre-trained LLM through tailored disentangled graph neural network layers. Tian et al. (2024) proposed graph neural prompts for encoding knowledge graphs into graph embeddings that LLMs can use during inference. In few-shot text-attributed graph learning, Huang et al. (2023b) introduced a prompt-based node feature extractor (G-Prompt) that combines a learnable GNN adapter with task-specific prompts to capture neighborhood information and extract informative node features.

Another projector-oriented line condenses graph information into fixed-length prefixes using graph transformers attached to LLM layers (Chai et al. 2023). Similarly, Yang et al. (2021) developed GraphFormers, which integrates text encoding and graph aggregation in an iterative workflow by nesting GNNs alongside transformer blocks. Additionally, Peng et al. (2024) developed ChatGraph, an interactive interface through which users interact with graphs in natural language; in that approach, graphs are represented as sequential paths, and the LLM is fine-tuned on graph data to support graph analysis.

In this paper, we do not consider projector adaptations that require retraining the LLM because they substantially increase computational cost without providing corresponding performance gains in the setting considered here. We instead focus on adaptation methods that use **frozen pre-trained**

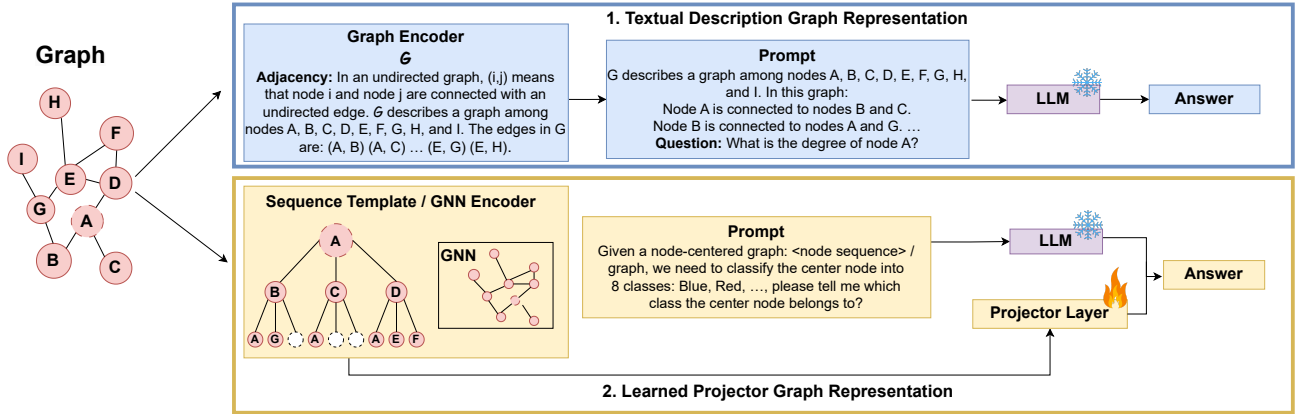


Figure 5: **Graph encoding-based adaptations of LLMs.** Textual-description approaches express graphs as natural language (top). Learned-projector approaches encode graphs through node templates that produce sequences or through GNN encoders (bottom). In both cases, the LLM is frozen, and only the projector that maps graph information into token-compatible representations is trained.

Table 7: Comparison of graph encoding-based adaptations of LLMs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

LLMs (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This design reduces computational overhead, supports integration with different LLM backbones, and preserves the knowledge encoded in the frozen model while adapting it to graph tasks. Throughout the paper, we use two representative projector methods: **LLAGA** (Chen et al. 2024), which uses node-level templates and a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which uses a GNN-based projector to encode the graph.

This work. We exclude textual-description methods from the empirical robustness study because their limitations are already tied to natural-language serialization and prompt scalability rather than to a learned graph encoding interface. These methods are generally suited to simpler tasks, such as describing basic graph properties like node degree, and become increasingly impractical for larger graphs because detailed graph descriptions can quickly exceed prompt token budgets. From a vulnerability perspective, textual-description methods also do not introduce the same graph-specific attack surface: when a graph is represented as text, such as "Node A is connected to Node B," the representation is exposed primarily to conventional text-based adversarial attacks rather than to the structural and feature perturbations characteristic of graph-based models. Projector-based graph-aware LLMs therefore provide the appropriate setting for studying adversarial robustness at the interface between graph structure, textual features, and frozen LLM representations.

A.2 Attacks on GNN

Several attack families have been studied for GNNs, including membership inference attacks (Olatunji, Nejdli, and Khosla 2021; He et al. 2021; Conti et al. 2022), which determine whether a node was part of the training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which infer sensitive or missing node attributes from model outputs or embeddings; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which extract global graph properties; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade performance through subtle perturbations of graph structure or node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which attempt to recover graph structure from learned embeddings or model outputs.

Adversarial attacks are the most directly relevant to this work because they target predictive integrity. Such attacks introduce small, carefully crafted *perturbations* into graph data, including feature modifications or edge changes, to degrade GNN performance. They are commonly categorized as **poisoning attacks**, which manipulate training data during the *training phase*, and **evasion attacks**, which introduce perturbations at *test time*. For instance, Zügner, Akbarnejad, and Günnemann (2018) showed that adversarial perturbations to graph structures or node features can substantially degrade GNN performance. Their method, **NETTACK**, uses greedy optimization to compute minimal perturbations that maximize the model’s training loss, identifying small

changes in features or structure that disrupt predictions. Li et al. (2023) revisited graph adversarial attacks from a data-distribution perspective, showing that adversarial examples can alter the underlying data distribution and proposing a defense to mitigate this effect. Similarly, Wu et al. (2019) analyzed adversarial examples on graph data and proposed defenses designed to detect and counteract adversarial modifications. **METAATTACK** (Zügner and Günnemann 2019) performs both poisoning and evasion attacks on GNNs by treating graph structure as an optimizable hyperparameter. The method directly modifies graph structure with gradient descent and formulates the attack as a bi-level optimization problem. Its meta-gradients capture how small structural perturbations affect the attacker’s loss after training, enabling targeted graph updates that degrade model performance.

In this paper, we use **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attack methods covering both poisoning and evasion strategies. Poisoning attacks compromise model integrity during training, whereas evasion attacks compromise performance at inference or test time.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs graph structure by adding or removing edges while preserving the degree distribution of the perturbed graph. It uses greedy optimization over a surrogate model to identify minimal perturbations that maximize the model’s training loss. The surrogate is a linearized 2-layer graph convolutional network (GCN), which approximates how structural changes affect classification. The attack increases confidence in incorrect classifications by maximizing the distance between logits produced on the perturbed and clean graphs. Operationally, **NETTACK** identifies candidate edges around a target node, evaluates a score function for each candidate edge, selects the highest-scoring perturbation, and updates the graph by adding or removing that edge. Because **NETTACK** is originally targeted, causing misclassification of a single specified node, we follow (Zügner and Günnemann 2019) to obtain an untargeted evaluation: each test node is attacked with **NETTACK** while considering all nodes in the network, and the resulting outcomes are aggregated over the test nodes.

METAATTACK optimizes graph structure directly through gradient descent by treating the adjacency matrix as a hyperparameter. It formulates the attack as a bi-level optimization problem: the inner level minimizes training loss on the original graph, and the outer level optimizes the attack objective on the poisoned graph. The method uses a linearized two-layer GCN surrogate to estimate how graph-structure changes affect model performance. A score function evaluates each candidate perturbation, and meta-gradients with respect to hyperparameters determine which connected edges should be removed. The algorithm then greedily selects the highest-scoring edge additions or removals to maximize attack impact.

For both methods, we use the unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which restricts changes to the graph’s degree distribution.

Accordingly, we perturb only 10% of the edges.

In poisoning experiments with LLAGA, the perturbed graph produced by NETTACK or METAATTACK is used to generate node-level sequences, and the projector that maps these sequences into token embeddings is retrained. For GRAPHPROMPTER, the perturbed graph is re-encoded with a graph attention network (GAT), and the projector is retrained on the perturbed graph. In evasion experiments, the graph-aware LLM is trained on unperturbed data, and perturbations are introduced only at inference time.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 specifies the node sequence template injection attack for LLAGA. The model constructs a *fixed-shape* computational tree for each node u by sampling k neighbors per node up to depth d (lines 1–14). When a node has fewer than k neighbors, placeholders are inserted to preserve the template shape, thereby creating the attack surface. We exploit this mechanism by inserting malicious nodes into placeholder positions through graph manipulation. The three strategies differ in the injected node type: *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* inserts one high-degree node into a placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across placeholders (lines 35–43). In each case, the injected nodes corrupt the target sequence representation and impair the projection layer.

A.5 Ablation Studies

This section reports the ablation results referenced in the main paper.

Ablation Results: Varying Feature Perturbation Figure 6 reports the accuracy of LLAGA and GRAPHPROMPTER as the percentage of perturbed features varies. Increasing the perturbation proportion consistently reduces accuracy for both models, confirming that feature-level attacks scale with the amount of corrupted textual input.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 show that the base LLM has minimal effect on clean performance and adversarial vulnerability in the LLAGA framework. Across Cora, Citeseer, and PubMed, Vicuna-7B and Llama2-7B variants achieve nearly identical clean accuracy and exhibit similar degradation under poisoning and evasion attacks. For example, under METAATTACK evasion on Cora, accuracy is 0.44 ± 0.06 for Vicuna-7B and 0.43 ± 0.08 for Llama2-7B, a difference within the reported standard deviations. Similar patterns hold across datasets and attack types, indicating that attack effectiveness in this setting is driven more by graph encoding and perturbation channel than by the specific base LLM.

A.6 Results on the ArXiv Dataset

We further evaluate node sequence template attacks on the large-scale ArXiv dataset, which contains 169,343 nodes,

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:   $G' \leftarrow G$  ▷ Create copy of original graph
17:   $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:   $P \leftarrow \text{GetPlaceholders}(T)$ 
19:  if  $|P| > 0$  then
20:     $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:    if  $\text{AttackStrategy} = \text{NI}$  then
22:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:       $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:      for each placeholder position  $p \in P$  do
25:         $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:        Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:      end for
28:    else if  $\text{AttackStrategy} = \text{SI}$  then
29:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:       $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:      for first placeholder position  $p \in P$  do
32:        Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:        break ▷ Only use first placeholder
34:      end for
35:    else if  $\text{AttackStrategy} = \text{MSI}$  then
36:       $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:       $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:      for  $i = 1$  to  $|P|$  do
39:         $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:         $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:        Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:      end for
43:    end if
44:  end if
45:  return  $G'$ 
46: end procedure

```

1,166,243 edges, and sparsity of 0.81. This experiment tests whether the injection vulnerability remains relevant on a substantially larger graph. As shown in Table 9, ArXiv exhibits a

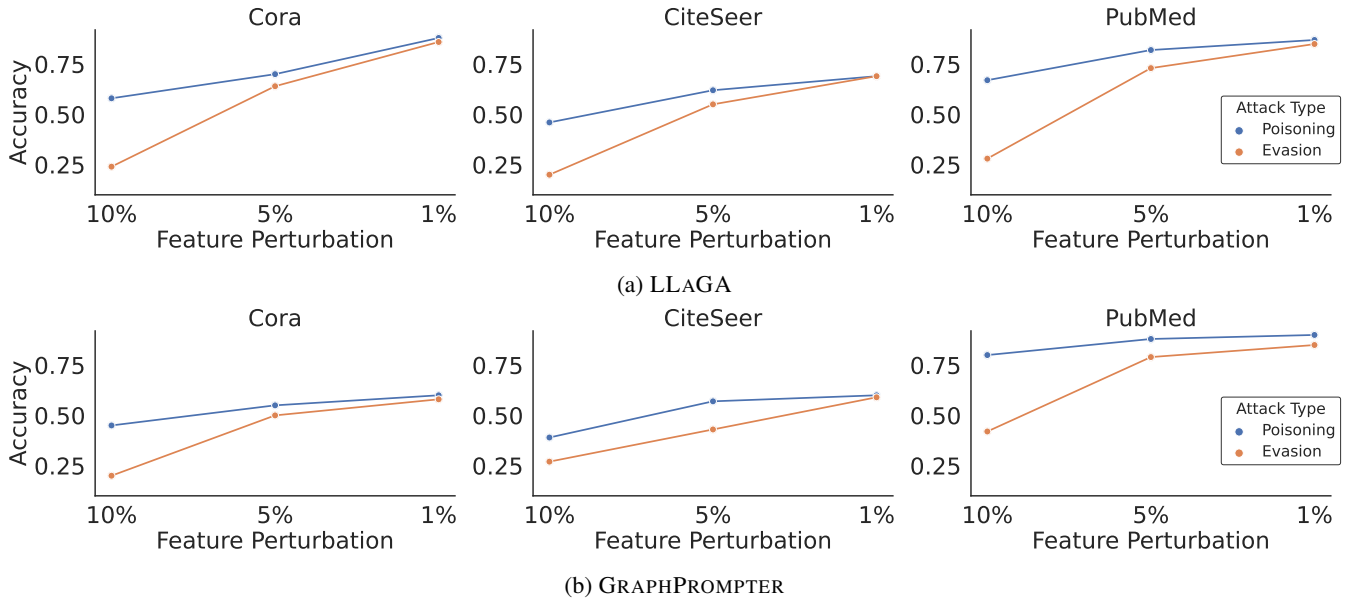


Figure 6: Accuracy of graph-aware LLMs as the percentage of perturbed features varies. Lower accuracy indicates a stronger attack.

Table 8: Impact of base LLM choice on LLAGA performance under clean, poisoning, and evasion settings. The results show that changing the base model does not substantially alter attack susceptibility.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

different pattern from Cora, Citeseer, and PubMed: poisoning attacks are consistently more damaging than evasion attacks across all three injection strategies. This result suggests that ArXiv’s structural properties make poisoning comparatively more effective while leaving evasion less disruptive. Despite the low sparsity of ArXiv, inserting a single supernode can still substantially affect model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples in Appendix A.7 illustrate adversarial feature perturbations applied to real Cora node features. The “input” is the text received by the LLM after the attack. Although a human reader perceives the original and perturbed text as identical or highly similar, the LLM processes different Unicode characters or a different character order, which can alter downstream behavior. Table 10 illustrates the Homoglyph attack, and Table 11 illustrates the Reordering attack.

Table 9: Accuracy under node sequence template injection attacks across datasets. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection attacks, respectively; Clean reports performance without attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	—	0.73 ± 0.02	—
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

Table 10: Imperceptible Homoglyph attack. The perturbed text is processed as distinct characters by the LLM while remaining visually nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtln squnc pptrns Abstract: Svrl cmputr lgrithms fr discvring pptrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f sttistical mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Imperceptible Reordering attack. The perturbed text changes the character order processed by the LLM while preserving a nearly identical visual rendering for a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Feature Corrector Defense Template We designed a feature corrector module to mitigate the effects of imperceptible attacks, specifically homoglyph substitutions and character reorderings. This module is implemented as a prompt-based defense using LLMs. The prompt is carefully crafted to instruct the LLM to act as a linguistic validator, focusing on detecting and correcting subtle, human-imperceptible perturbations.

The template for our prompt is shown below. It explicitly states the task and provides a clear output format, which significantly improves the LLM’s performance.

A.8 Detailed Discussion

The results provide several concrete insights into the vulnerability profile and design tradeoffs of graph-aware LLMs. As graph-structured data are increasingly integrated with

LLMs for graph tasks, robustness must be evaluated alongside predictive performance. Our systematic investigation of two representative graph-aware LLMs, LLAGA and GRAPH-PROMPTER, shows that poisoning and evasion attacks developed for GNNs remain relevant, while graph-aware LLMs also introduce additional vulnerabilities through their encoding interfaces. The key observations are discussed below.

Impact of Graph Encoding on Adversarial Vulnerability

A central observation is that the method used to integrate graph structure into an LLM can itself create adversarial susceptibility. The node sequence template in LLAGA creates an exploitable vulnerability that produces substantial performance degradation. This LLAGA-specific attack surface highlights a design tradeoff: node-level templates provide an effective mechanism for incorporating graph structure, but they also expose attack vectors that do not arise in

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgpprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly allcot th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llction plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: Prompt template for the LLM-based feature corrector. The prompt specifies the model role, correction task, and expected output format.

conventional GNNs. By contrast, GRAPHPROMPTER uses a GNN-based projector for graph encoding and demonstrates comparatively stronger resilience, suggesting that hybrid architectures can benefit from separating textual representation from structurally perturbed graph components.

Sensitivity of Graph-aware LLMs to Feature Perturbations The experiments also establish feature perturbation as a major vulnerability across both graph-aware LLMs. Despite their different structural encoders, LLAGA and GRAPH-PROMPTER are both sensitive to imperceptible changes in textual node features. This behavior reflects a broader LLM robustness challenge: models that are effective at processing text can remain vulnerable to adversarial character-level manipulations. The ability to degrade graph-aware LLM performance by subtly altering node attributes without clear human-visible changes raises integrity concerns, especially for security-sensitive uses. Text sanitization techniques such as those discussed by (Boucher et al. 2022) provide one possible direction, but imperceptible attacks such as homoglyph manipulations remain difficult to eliminate. The LLM corrector in GALGUARD addresses this vulnerability by prompting a model to detect and repair suspicious or malformed inputs before downstream graph-aware processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Attack effectiveness is not uniform across datasets. Cora, Citeseer, and PubMed show strong vulnerability patterns in the main experiments, whereas ArXiv exhibits a more nuanced interaction between attack strategy and graph structure. This variation shows that dataset characteristics are essential to robustness evaluation. As graph-aware LLMs are applied across domains, their vulnerabilities should be assessed not only through model design but also through the structural and textual properties of the data on which they operate.

Implications for Graph-aware LLM Development These vulnerabilities have direct implications for the development and deployment of graph-aware LLMs. Because these models increasingly target tasks historically addressed by GNNs, their design must account for the tradeoff between expressive graph encoding and adversarial exposure. Future graph-aware LLMs require more robust encoding mechanisms and defenses that jointly address structural and textual feature perturbations. Understanding how GNN attacks transfer to graph-aware LLMs also opens a concrete research agenda for protecting these models against more sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have predominantly focused on structural perturbations because feature-based perturbations typically fail to change the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019). Consequently, there are *no known defenses against feature-based attacks on GNNs* motivated by strong feature-attack effectiveness. The main results show that this situation changes in the graph-aware LLM

paradigm: feature perturbations that are weak in conventional GNN settings can successfully manipulate model behavior once textual node attributes are processed by an LLM. This shift requires defenses that address both structural and feature-based adversarial perturbations. Text sanitization techniques, including optical character recognition (OCR) as a preprocessing step before LLM encoding, may help in some cases, but imperceptible attacks such as homoglyph manipulations remain difficult because OCR is imperfect and tends to misinterpret homoglyphs at a higher rate than unperturbed text (Boucher et al. 2022). This limitation motivates the LLM-based self-correction mechanism used for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based adversarial attacks transfer from GNNs to graph-aware LLMs, particularly under evasion, it is natural to ask whether GNN defenses transfer as well. The results indicate that this assumption is insufficient. Unlike traditional GNN settings, *graph-aware LLMs are also susceptible to feature perturbations, rendering these defenses ineffective* when they rely on clean textual attributes.

One GNN defense mechanism makes the adjacency matrix trainable so that edge weights can adapt during training to mitigate adversarial perturbations. This approach assigns lower weights to edges connecting dissimilar nodes and higher weights to edges connecting nodes with greater feature similarity. Several **preprocessing-based** defenses build on this idea, including removing edges between nodes with low feature similarity (Wu et al. 2019) and randomized edge dropping (Dai et al. 2018). These defenses, however, depend on clean node features to compute reliable similarity scores. When features are also perturbed, as in our setting, similarity-based defenses can become ineffective and may even worsen the graph by introducing incorrect edges. We illustrate this failure mode on a random node in Cora (Figure 8). The clean feature-similarity graph preserves some original edges while introducing some incorrect connections, whereas the graph reconstructed from perturbed features preserves no original connections, producing an entirely incorrect reconstruction.

Another defense line reduces reliance on graph structure. For instance, (Shanthamallu, Thiagarajan, and Spanias 2021) proposed training a surrogate predictor that depends only on node features and uses uncertainty matching to extract graph information from the GNN. Similarly, Wu et al. (2021) introduced a co-training framework that trains submodels independently on feature and structural views and distills knowledge by adding confident unlabeled data to the training set. These approaches also assume access to unperturbed node features, which is not available when both structure and features are adversarially perturbed. Structural defenses developed for GNNs therefore do not directly solve robustness for graph-aware LLMs, motivating defenses tailored to this paradigm.

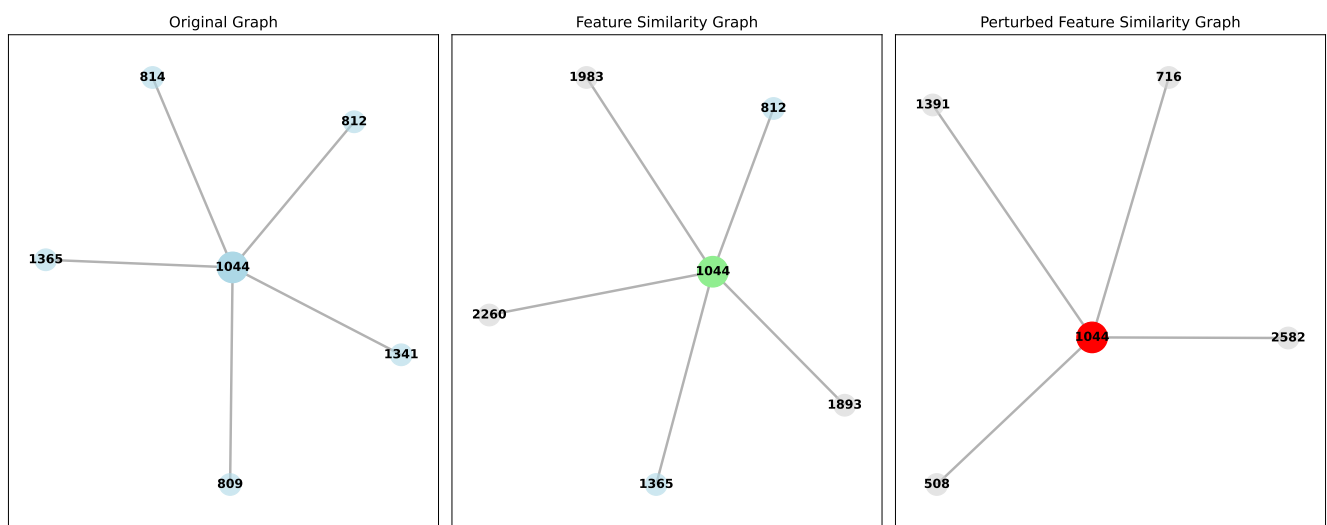


Figure 8: Edges reconstructed for random node 1044. **Original (left)** shows ground-truth edges from the original graph. **Feature Similarity (middle)** shows edges reconstructed from clean node features, with blue nodes indicating preserved original connections. **Perturbed Feature Similarity (right)** shows edges reconstructed from perturbed node features.